PENSE-BÊTE · TECHNIQUE ET COMMANDES

Gestion des processus Linux

06-820 PSE 25-27 SP1-6 Linux - Les processus — source : fiche de révision du module (gemma4, 03/11/2026)

À RETENIR

1. Le processus est un programme en cours d'exécution possédant son propre compteur ordinal et ses ressources.
2. Le processus init de PID 1 est l'ancêtre de tous les autres processus et accueille les processus orphelins.
3. La commutation de contexte est une opération coûteuse consistant à sauvegarder les informations du processus évincé et à charger celles du processus élu.

Fondamentaux des processus

Notion	À retenir
Processus vs Programme	Le **processus** est un programme en cours d'exécution (entité active avec compteur ordinal et ressources). Le **programme** est l'ensemble des fichiers exécutés.
Bloc de contrôle (PCB)	Stocke informations et ressources. UNIX : `struct proc` (`<sys/proc.h>`). Linux : `task_struct`. Inclut `struct user` (zone `u`) et éléments comme le **compteur ordinal**.
Multiprogrammation	Capacité de faire tourner plusieurs tâches. Sur un seul processeur : **pseudoparallélisme** (commutation rapide) créant l'illusion de simultanéité.
Typologie des processus	Utilisateurs : priorité moindre, mode utilisateur. Démons : sans terminal. Systèmes : exécution du **noyau**, priorité élevée.

Cycle de vie et gestion des processus

Notion	À retenir
Mécanisme fork	Appel système `fork(void)` : crée un processus **fils** à partir d'un processus **père**. Chaque processus a **un seul père**.
Hiérarchie et init	PID=**0** (système) crée PID=**1** (`init`, ancêtre de tous) et PID=**2** (`kthreadd`, parent des threads noyau). `init` recueille les **orphelins**.
Signaux et kill	Communication par entiers via `/usr/include/signal.h`. `kill sig pid` envoie le signal. `CTRL-C` envoie `SIGINT`. Si `sig` est nul, succès si `pid` existe.
États et modes	États transitoires : **Initialisation** et **Zombie**. Exécution → prêt = **changement de contexte** ; mode noyau → mode utilisateur = **changement de mode**. Mis en **SWAP** en priorité : les processus **endormis**.

Commutation, interruptions et priorité

Notion	À retenir
Commutation de contexte	Opération coûteuse (**overhead**) déclenchée par le `scheduler` lors de l'épuisement du temps ou d'une attente **E/S**. Sauvegarde du contexte (registres, mémoire) du processus évincé et chargement de l'élu.
Interruptions et priorité	Si niveau interruption **>** processus : interruption traitée immédiatement. Si niveau interruption **<** processus : interruption mise en attente. Noyau Linux **préemptible** (depuis **2.6**, préemption temps réel **PREEMPT_RT** depuis **6.12**).
Influence sur la priorité	Seule la valeur de **nice** influe sur un processus utilisateur. `nice -n [niveau] [programme]` définit la priorité à la création. `renice -n [niveau] [PID]` modifie un processus déjà en cours.

Structure de la mémoire des processus

Notion	À retenir
Éléments du contexte	Regroupe les infos nécessaires à l'exécution (**mot d'état**) : compteur ordinal (adresse instruction), registre d'état, pointeur de pile, registres généraux et registre de l'espace mémoire.
Table des processus	Contient une entrée par processus. Taille dynamique sous Linux (limites `pid_max` et `threads-max`). Inclut **PID**, **PPID**, **RUID**, **EUID**, **RGID**, **EGID**, et descripteurs de fichiers.
Régions de mémoire	Table privée de pointeurs vers l'adresse virtuelle. Régions de **code** (partagées), de **données** (partagées ou privées) et de **pile** (strictement privées).
Interface `/proc`	Pseudo système de fichiers en mémoire. Sous-répertoires `/proc/PID` contenant : `cmdline` (commande), `cwd` (répertoire courant), `environ` (variables), `exe` (exécutable), `fd` (fichiers/sockets) et `maps` (cartographie).

Ordonnancement et gestion du processeur

Notion	À retenir
Objectifs et enjeux	Décider quel processus utilise le CPU, quand et combien de temps. Assure la réactivité, le multitâche et évite la monopolisation par un processus.
Partage et risques	Respecter l'**exclusion mutuelle** (ressource à un unique processus). Éviter la **famine** (accès refusé un temps indéterminé) en garantissant un temps fini.
Priorités et quantum	Priorité élevée si valeur numérique faible (min **0**). Partage par **quantum** (`timeslice`). Priorité utilisateur via valeur **nice**. Noyau/temps réel : **0-99** ; utilisateur : **100-139**.
Préemption et scheduler	Le `scheduler` (**PID 0**) gère l'ordonnancement. Préemption en fin de quantum ou retour mode noyau. Sous Linux, le noyau est **préemptible**.

Pièges et confusions

- ⚠ Confondre un programme et un processus : le processus possède l'état courant en mémoire alors que le programme n'est qu'un ensemble de fichiers.
- ⚠ Confondre le processus 0 et le processus 1 : le premier est créé manuellement au démarrage et le second est l'ancêtre de tous les autres.
- ⚠ Confondre la préemption et l'interruption : dans le modèle UNIX classique, un processus en mode noyau peut être interrompu mais pas préempté.
- ⚠ Erreur sur la priorité : la priorité d'un processus varie en sens inverse de sa valeur numérique.
- ⚠ Confondre le mode utilisateur et le mode noyau : un processus utilisateur peut effectuer des appels système pour passer en mode noyau.

Sigles et définitions

PCB	structure qui stocke l'ensemble des ressources et l'état courant d'un processus
PID	identifiant numérique unique attribué à chaque processus
Signal	mécanisme de communication entre processus identifié par un nombre entier
Scheduler	ordonnanceur interne chargé de déclencher la commutation de processus
Nice	paramètre permettant d'influer manuellement sur la priorité d'un processus utilisateur
Exclusion mutuelle	principe consistant à affecter une ressource à un unique processus à la fois

Auto-test

1. Quel est l'identifiant du processus qui est l'ancêtre de tous les autres ?
2. Comment appelle-t-on la capacité d'un système à faire tourner plusieurs tâches à la fois via le pseudoparallélisme ?
3. Quel répertoire sert d'interface entre le noyau et l'utilisateur pour obtenir des informations sur les processus ?
4. Quel signal est engendré par la commande CTRL-C ?
5. Quelle commande permet de modifier la priorité d'un processus déjà en cours ?

Réponses : **1.** 1 · **2.** La multiprogrammation · **3.** /proc · **4.** SIGINT · **5.** renice

À compléter par la fiche, le podcast et le quiz du module. Rédigé à partir de la fiche de révision ; seuils et chiffres à la date de la source.